Programming Portfolio

Kevin Tieu

Selected Projects
October 2026

Table of Contents

Technique Table ii
Lockbox 1
GraphNet 30
Traffic Light 38
Multi-Connection Chat 42

Technique Table

Each program is listed with the primary techniques it demonstrates.

Project	CLI Args	File I/O	Process	Threads	Algorithms	Testing
Lockbox	X	X	X			
GraphNet					X	
Traffic Light						X
Multi-Connection Chat				X		

Color key: Orange = personal / other Red = CS 346

Lockbox

Command-Line Utility

A multi-feature command-line utility. It dispatches commands, copies files, handles environment variables, maintains a binary index, logs activity, controls processes, and offers a screen-based file browser. This project is still ongoing.

main.c

```
/*
 * main.c
 * CS 375 – lockbox
 *
 * Author: Kevin Tieu
 *
 * Entry point. Parse the command line and call one command.
 */

#define _POSIX_C_SOURCE 200809L

#include "lockbox.h"
#include "env.h"
#include "vault.h"
#include "commands.h"
#include "screen.h"
#include "index.h"
#include "process.h"

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <getopt.h>

/*
 * main
 * Start the program. Read the command line and run one command.
 *
 * argument_count - how many words are on the command line
 * argument_list  - those words. argument_list[0] is the program name
 * return 0 if it worked, 1 if it failed
 */
int main(int argument_count, char *argument_list[])
{
    int option_letter;
    int first_command_index;

    apply_start_settings();
    install_interrupt_handler();

    opterr = 0;
    option_letter = getopt(argument_count, argument_list, "d:");
    while (option_letter != -1) {
        if (option_letter == 'd') {
            set_vault_directory(optarg);
        } else {
            print_usage(argument_list[0]);
            return 1;
        }
        option_letter = getopt(argument_count, argument_list, "d:");
    }
    first_command_index = optind;
    if (first_command_index >= argument_count) {
        print_usage(argument_list[0]);
        return 1;
    }
    if (ensure_vault() != 0) {
        return 1;
    }
    if (strcmp(argument_list[first_command_index], "put") == 0) {
        if (first_command_index + 1 >= argument_count) {
            print_usage(argument_list[0]);
            return 1;
```

```
        }
        return command_put(argument_list[first_command_index + 1]);
    }
    if (strcmp(argument_list[first_command_index], "get") == 0) {
        if (first_command_index + 2 >= argument_count) {
            print_usage(argument_list[0]);
            return 1;
        }
        return command_get(argument_list[first_command_index + 1],
                           argument_list[first_command_index + 2]);
    }
    if (strcmp(argument_list[first_command_index], "list") == 0) {
        return command_list();
    }
    if (strcmp(argument_list[first_command_index], "env") == 0) {
        return command_env();
    }
    if (strcmp(argument_list[first_command_index], "screen") == 0) {
        return command_screen();
    }
    if (strcmp(argument_list[first_command_index], "index") == 0) {
        return command_index();
    }
    fprintf(stderr, "unknown command: %s\n",
            argument_list[first_command_index]);
    print_usage(argument_list[0]);
    return 1;
}
```

lockbox.h

```
/*
 * lockbox.h
 * CS 375 – lockbox
 *
 * Author: Kevin Tieu
 *
 * Sizes and names shared by every .c file.
 */

#ifndef LOCKBOX_H
#define LOCKBOX_H

#define DEFAULT_VAULT_DIR "vault"  /* folder used when LOCKBOX_DIR is not set */
#define COPY_BUFFER_SIZE 4096  /* bytes copied in each read/write step */
#define PATH_BUFFER_SIZE 512  /* size of a full path string */
#define NAME_BUFFER_SIZE 256  /* size of a stored file name */
#define LINE_BUFFER_SIZE 256  /* size of one input line */
#define LOG_FILE_NAME "lockbox.log"  /* text log inside the vault */
#define LOG_LINE_SIZE 640  /* size of one log line */
#define INDEX_FILE_NAME "index.txt"  /* text index inside the vault */
#define INDEX_BIN_NAME "index.bin"  /* binary index inside the vault */
#define SCREEN_MAX_FILES 128  /* names the file list can show */

#endif
```

commands.c

```
/*
 * commands.c
 * CS 375 — lockbox
 *
 * Author: Kevin Tieu
 *
 * put copies a file in. get copies it out. list names what is stored.
 */

#include "commands.h"
#include "lockbox.h"
#include "vault.h"
#include "env.h"
#include "log.h"
#include "copy.h"
#include "index.h"

#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <dirent.h>

/*
 * print_usage
 * How to run this program. Writes to stderr.
 *
 * program_name - argument_list[0]
 * return nothing
 */
void print_usage(const char *program_name)
{
    fprintf(stderr, "usage:\n");
    fprintf(stderr, "  %s put <file>\n", program_name);
    fprintf(stderr, "  %s get <name> <outfile>\n", program_name);
    fprintf(stderr, "  %s list\n", program_name);
    fprintf(stderr, "  %s env\n", program_name);
    fprintf(stderr, "  %s screen\n", program_name);
    fprintf(stderr, "  %s index\n", program_name);
    fprintf(stderr, "  %s -d DIR <command>\n", program_name);
}

/*
 * command_put
 * Copy source_path into the vault under its base name.
 * O_EXCL refuses to overwrite a name that is already stored.
 *
 * source_path - file the user wants to store
 * return 0 if it worked, 1 if it failed
 */
int command_put(const char *source_path)
{
    int input_descriptor;
    int output_descriptor;
    char vault_path[PATH_BUFFER_SIZE];
    const char *stored_name;
    char log_line[LOG_LINE_SIZE];
    struct stat file_info;

    stored_name = base_name(source_path);
    if (make_vault_path(vault_path, sizeof(vault_path), stored_name) != 0) {
        return 1;
```

```
    }
    input_descriptor = open(source_path, O_RDONLY);
    if (input_descriptor < 0) {
        perror(source_path);
        return 1;
    }
    output_descriptor = open(vault_path, O_WRONLY | O_CREAT | O_EXCL, 0600);
    if (output_descriptor < 0) {
        perror(vault_path);
        close(input_descriptor);
        return 1;
    }
    if (copy_descriptor(input_descriptor, output_descriptor) != 0) {
        close(input_descriptor);
        close(output_descriptor);
        return 1;
    }
    close(input_descriptor);
    close(output_descriptor);
    if (stat(vault_path, &file_info) == 0) {
        index_record_put(stored_name, (long)file_info.st_size);
    }
    snprintf(log_line, sizeof(log_line), "PUT %s -> %s",
             source_path, vault_path);
    log_action(log_line);
    printf("stored %s\n", vault_path);
    return 0;
}

/*
 * command_get
 * Copy a stored file out to output_path.
 *
 * stored_name - name inside the vault
 * output_path - file to write on the outside
 * return 0 if it worked, 1 if it failed
 */
int command_get(const char *stored_name, const char *output_path)
{
    int input_descriptor;
    int output_descriptor;
    char vault_path[PATH_BUFFER_SIZE];
    char log_line[LOG_LINE_SIZE];

    if (make_vault_path(vault_path, sizeof(vault_path), stored_name) != 0) {
        return 1;
    }
    input_descriptor = open(vault_path, O_RDONLY);
    if (input_descriptor < 0) {
        perror(vault_path);
        return 1;
    }
    output_descriptor = open(output_path, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (output_descriptor < 0) {
        perror(output_path);
        close(input_descriptor);
        return 1;
    }
    if (copy_descriptor(input_descriptor, output_descriptor) != 0) {
        close(input_descriptor);
        close(output_descriptor);
        return 1;
    }
    close(input_descriptor);
    close(output_descriptor);
    snprintf(log_line, sizeof(log_line), "GET %s -> %s",
             stored_name, output_path);
    log_action(log_line);
```

```
    printf("wrote %s\n", output_path);
    return 0;
}

/*
 * command_list
 * Walk the vault folder with opendir and readdir.
 *
 * return 0 if it worked, 1 if the folder would not open
 */
int command_list(void)
{
    DIR *vault_folder;
    struct dirent *entry;
    int file_count;

    vault_folder = opendir(vault_directory());
    if (vault_folder == NULL) {
        perror(vault_directory());
        return 1;
    }
    file_count = 0;
    entry = readdir(vault_folder);
    while (entry != NULL) {
        if (!is_bookkeeping_name(entry->d_name)) {
            printf("%s\n", entry->d_name);
            file_count = file_count + 1;
        }
        entry = readdir(vault_folder);
    }
    closedir(vault_folder);
    log_action("LIST");
    printf("(%d file%s)\n", file_count, file_count == 1 ? "" : "s");
    return 0;
}
```

commands.h

```
/*
 * commands.h
 * CS 375 — lockbox
 *
 * Author: Kevin Tieu
 *
 * put, get, list, and the usage text.
 */

#ifndef COMMANDS_H
#define COMMANDS_H

/*
 * print_usage
 * How to run this program. Writes to stderr.
 *
 * program_name - argument_list[0]
 */
void print_usage(const char *program_name);

/*
 * command_put
 * Copy source_path into the vault under its base name.
 *
 * source_path - file the user wants to store
 * return 0 if it worked, 1 if it failed
 */
int command_put(const char *source_path);

/*
 * command_get
 * Copy a stored file out to output_path.
 *
 * stored_name - name inside the vault
 * output_path - file to write on the outside
 * return 0 if it worked, 1 if it failed
 */
int command_get(const char *stored_name, const char *output_path);

/*
 * command_list
 * Walk the vault folder and print stored names.
 *
 * return 0 if it worked, 1 if the folder would not open
 */
int command_list(void);

#endif
```

copy.c

```
/*
 * copy.c
 * CS 375 – lockbox
 *
 * Author: Kevin Tieu
 *
 * Copy bytes between two open file descriptors.
 */

#include "copy.h"
#include "lockbox.h"

#include <stdio.h>
#include <unistd.h>

/*
 * copy_descriptor
 * Read from input_descriptor and write to output_descriptor
 * until the input ends. write may send fewer bytes than asked,
 * so the inner loop keeps going until this chunk is done.
 *
 * input_descriptor  - open file we read
 * output_descriptor - open file we write
 * return 0 if it worked, 1 if a read or write failed
 */
int copy_descriptor(int input_descriptor, int output_descriptor)
{
    char buffer[COPY_BUFFER_SIZE];
    ssize_t bytes_read;
    ssize_t bytes_written;
    ssize_t bytes_left;
    char *write_position;

    /* Read the first chunk */
    bytes_read = read(input_descriptor, buffer, sizeof(buffer));
    while (bytes_read > 0) {
        write_position = buffer;
        bytes_left = bytes_read;
        while (bytes_left > 0) {
            bytes_written = write(output_descriptor, write_position,
                                  (size_t)bytes_left);
            if (bytes_written < 0) {
                perror("write");
                return 1;
            }
            bytes_left = bytes_left - bytes_written;
            write_position = write_position + bytes_written;
        }
        bytes_read = read(input_descriptor, buffer, sizeof(buffer));
    }
    if (bytes_read < 0) {
        perror("read");
        return 1;
    }
    return 0;
}
```

copy.h

```
/*
 * copy.h
 * CS 375 – lockbox
 *
 * Author: Kevin Tieu
 *
 * Copy bytes between two open file descriptors.
 */

#ifndef COPY_H
#define COPY_H

/*
 * copy_descriptor
 * Read from input_descriptor and write to output_descriptor
 * until the input ends.
 *
 * input_descriptor  - open file we read
 * output_descriptor - open file we write
 * return 0 if it worked, 1 if a read or write failed
 */
int copy_descriptor(int input_descriptor, int output_descriptor);

#endif
```

env.c

```
/*
 * env.c
 * CS 375 — lockbox
 *
 * Author: Kevin Tieu
 *
 * getenv, umask, and the -d folder override.
 */

#include "env.h"
#include "lockbox.h"

#include <stdlib.h>
#include <sys/stat.h>
#include <stdio.h>

static const char *override_directory = NULL;

/*
 * vault_directory
 * Use the -d folder if it was set. Otherwise use LOCKBOX_DIR.
 * If neither is set, use the default vault folder.
 *
 * return pointer to that folder name
 */
const char *vault_directory(void)
{
    const char *from_environment;

    if (override_directory != NULL && override_directory[0] != '\0') {
        return override_directory;
    }
    from_environment = getenv("LOCKBOX_DIR");
    if (from_environment != NULL && from_environment[0] != '\0') {
        return from_environment;
    }
    return DEFAULT_VAULT_DIR;
}

/*
 * apply_start_settings
 * Set umask so new files are not group or world writable
 * unless we pass a mode to open or mkdir.
 */
/*
 * apply_start_settings
 * Set the process umask so new files are not group or world writable
 * unless a mode is passed to open or mkdir.
 *
 * return nothing
 */
void apply_start_settings(void)
{
    umask(077);
}

/*
 * set_vault_directory
 * Remember a folder from the -d switch for this run.
 *
 * folder - path typed after -d
 * return nothing
```

```
 */
void set_vault_directory(const char *folder)
{
    override_directory = folder;
}

/*
 * command_env
 * Print USER, HOME, LOCKBOX_DIR, and the vault folder in use.
 *
 * return 0
 */
int command_env(void)
{
    const char *home;
    const char *user;
    const char *from_environment;

    home = getenv("HOME");
    user = getenv("USER");
    from_environment = getenv("LOCKBOX_DIR");
    printf("USER=%s\n", user != NULL ? user : "(not set)");
    printf("HOME=%s\n", home != NULL ? home : "(not set)");
    printf("LOCKBOX_DIR=%s\n",
           from_environment != NULL ? from_environment : "(not set)");
    printf("vault in use=%s\n", vault_directory());
    return 0;
}
```

env.h

```
/*
 * env.h
 * CS 375 – lockbox
 *
 * Author: Kevin Tieu
 *
 * Vault folder from the environment and the -d switch.
 */

#ifndef ENV_H
#define ENV_H

/*
 * vault_directory
 * Pick the folder that holds stored files.
 * LOCKBOX_DIR wins if it is set, unless -d was used.
 * return pointer to that folder name
 */
const char *vault_directory(void);
/*
 * apply_start_settings
 * Set umask so new files are not group or world writable
 * unless we pass a mode to open or mkdir.
 */
void apply_start_settings(void);
/*
 * set_vault_directory
 * Remember a folder from the -d switch for this run.
 *
 * folder - path typed after -d
 */
void set_vault_directory(const char *folder);
/*
 * command_env
 * Print USER, HOME, LOCKBOX_DIR, and the vault folder in use.
 * return 0
 */
int command_env(void);

#endif
```

index.c

```
/*
 * index.c
 * CS 375 — lockbox
 *
 * Author: Kevin Tieu
 *
 * Text index plus binary records and a file lock.
 */

#include "index.h"
#include "lockbox.h"
#include "vault.h"

#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>

struct vault_record {
    char file_name[NAME_BUFFER_SIZE];
    long file_size;
    time_t stored_time;
};

/*
 * lock_descriptor
 * Lock the index file so only one process writes at a time.
 *
 * file_descriptor - open index file
 * return 0 if the lock was taken, -1 if it failed
 */
static int lock_descriptor(int file_descriptor)
{
    struct flock lock;

    memset(&lock, 0, sizeof(lock));
    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    return fcntl(file_descriptor, F_SETLKW, &lock);
}

/*
 * unlock_descriptor
 * Release the lock on the index file.
 *
 * file_descriptor - open index file
 * return 0 if the lock was released, -1 if it failed
 */
static int unlock_descriptor(int file_descriptor)
{
    struct flock lock;

    memset(&lock, 0, sizeof(lock));
    lock.l_type = F_UNLCK;
    lock.l_whence = SEEK_SET;
    return fcntl(file_descriptor, F_SETLK, &lock);
}

/*
 * index_record_put
 * Append a text line and a binary record.
```

```
 *
 * stored_name - name inside the vault
 * file_size   - byte count
 * return 0 if it worked, 1 if a write failed
 */
int index_record_put(const char *stored_name, long file_size)
{
    FILE *text_file;
    int binary_descriptor;
    char text_path[PATH_BUFFER_SIZE];
    char binary_path[PATH_BUFFER_SIZE];
    struct vault_record record;

    if (make_index_path(text_path, sizeof(text_path)) != 0) {
        return 1;
    }
    if (make_index_bin_path(binary_path, sizeof(binary_path)) != 0) {
        return 1;
    }
    text_file = fopen(text_path, "a");
    if (text_file == NULL) {
        perror(text_path);
        return 1;
    }
    fprintf(text_file, "%s %ld\n", stored_name, file_size);
    fclose(text_file);

    memset(&record, 0, sizeof(record));
    strncpy(record.file_name, stored_name, NAME_BUFFER_SIZE - 1);
    record.file_size = file_size;
    record.stored_time = time(NULL);

    binary_descriptor = open(binary_path, O_WRONLY | O_CREAT | O_APPEND, 0600);
    if (binary_descriptor < 0) {
        perror(binary_path);
        return 1;
    }
    if (lock_descriptor(binary_descriptor) == -1) {
        perror("fcntl lock");
        close(binary_descriptor);
        return 1;
    }
    if (write(binary_descriptor, &record, sizeof(record)) !=
        (ssize_t)sizeof(record)) {
        perror("write index.bin");
        close(binary_descriptor);
        return 1;
    }
    unlock_descriptor(binary_descriptor);
    close(binary_descriptor);
    return 0;
}

/*
 * command_index
 * Print every binary record.
 *
 * return 0
 */
int command_index(void)
{
    int binary_descriptor;
    char binary_path[PATH_BUFFER_SIZE];
    struct vault_record record;
    ssize_t bytes_read;

    if (make_index_bin_path(binary_path, sizeof(binary_path)) != 0) {
```

```
        return 1;
    }
    binary_descriptor = open(binary_path, O_RDONLY);
    if (binary_descriptor < 0) {
        printf("no binary index yet\n");
        return 0;
    }
    bytes_read = read(binary_descriptor, &record, sizeof(record));
    while (bytes_read == (ssize_t)sizeof(record)) {
        printf("%s %ld\n", record.file_name, record.file_size);
        bytes_read = read(binary_descriptor, &record, sizeof(record));
    }
    close(binary_descriptor);
    return 0;
}
```

index.h

```
/*
 * index.h
 * CS 375 — lockbox
 *
 * Author: Kevin Tieu
 *
 * On-disk records of stored files.
 */

#ifndef INDEX_H
#define INDEX_H

/*
 * index_record_put
 * Remember that stored_name was put, and how big it is.
 *
 * stored_name - name inside the vault
 * file_size   - byte count
 * return 0 if it worked, 1 if the index could not be written
 */
int index_record_put(const char *stored_name, long file_size);

/*
 * command_index
 * Print every binary record.
 *
 * return 0
 */
int command_index(void);

#endif
```

log.c

```
/*
 * log.c
 * CS 375 – lockbox
 *
 * Author: Kevin Tieu
 *
 * Open the log with fopen and append one line.
 */

#include "log.h"
#include "lockbox.h"
#include "vault.h"

#include <stdio.h>

/*
 * log_action
 * Append one line to vault/lockbox.log.
 * "a" keeps old lines. A failed log write does not undo the command.
 *
 * message - text to store
 * return nothing
 */
void log_action(const char *message)
{
    FILE *log_file;
    char log_path[PATH_BUFFER_SIZE];

    if (make_log_path(log_path, sizeof(log_path)) != 0) {
        return;
    }
    log_file = fopen(log_path, "a");
    if (log_file == NULL) {
        perror(log_path);
        return;
    }
    fprintf(log_file, "%s\n", message);
    fclose(log_file);
}
```

log.h

```
/*
 * log.h
 * CS 375 – lockbox
 *
 * Author: Kevin Tieu
 *
 * Append-only text log inside the vault.
 */

#ifndef LOG_H
#define LOG_H

/*
 * log_action
 * Append one line to vault/lockbox.log.
 * A failed log write does not undo the command.
 *
 * message - text to store
 */
void log_action(const char *message);

#endif
```

process.c

```c
/*
 * process.c
 * CS 375 – lockbox
 *
 * Author: Kevin Tieu
 *
 * status uses fork. hash uses a pipe. SIGINT stops a blocked wait.
 */

#include "process.h"
#include "lockbox.h"
#include "vault.h"
#include "env.h"
#include "log.h"

#include <stdio.h>
#include <string.h>
#include <signal.h>
#include <unistd.h>

/*
 * handle_interrupt
 * Empty handler for SIGINT. The call that was blocked returns
 * so the program can exit cleanly.
 *
 * signal_number - the signal that arrived (unused)
 */
static void handle_interrupt(int signal_number)
{
    (void)signal_number;
}

/*
 * install_interrupt_handler
 * Catch SIGINT so a blocked read or accept can return.
 */
void install_interrupt_handler(void)
{
    signal(SIGINT, handle_interrupt);
}
```

process.h

```
/*
 * process.h
 * CS 375 – lockbox
 *
 * Author: Kevin Tieu
 *
 * Child processes, a pipe, and SIGINT.
 */

#ifndef PROCESS_H
#define PROCESS_H
void install_interrupt_handler(void);

#endif
```

screen.c

```
/*
 * screen.c
 * CS 375 – lockbox
 *
 * Author: Kevin Tieu
 *
 * Terminal file list of stored names.
 */

#include "screen.h"
#include "lockbox.h"
#include "vault.h"
#include "env.h"

#include <ncurses.h>
#include <string.h>
#include <dirent.h>

/*
 * collect_stored_names
 * Fill names with stored file names. Skip helper files.
 *
 * names     - caller array of name buffers
 * max_names - how many names fit
 * return how many names were stored
 */
static int collect_stored_names(char names[][NAME_BUFFER_SIZE], int max_names)
{
    DIR *vault_folder;
    struct dirent *entry;
    int count;

    count = 0;
    vault_folder = opendir(vault_directory());
    if (vault_folder == NULL) {
        return 0;
    }
    entry = readdir(vault_folder);
    while (entry != NULL && count < max_names) {
        if (!is_bookkeeping_name(entry->d_name)) {
            strncpy(names[count], entry->d_name, NAME_BUFFER_SIZE - 1);
            names[count][NAME_BUFFER_SIZE - 1] = '\0';
            count = count + 1;
        }
        entry = readdir(vault_folder);
    }
    closedir(vault_folder);
    return count;
}

/*
 * draw_file_list
 * Draw the title and each stored name. The highlighted row is reversed.
 *
 * names            - stored names
 * file_count       - how many names
 * highlight_index  - row the user is on
 */
static void draw_file_list(char names[][NAME_BUFFER_SIZE], int file_count,
                           int highlight_index)
{
    int row;
```

```
    clear();
    mvprintw(0, 0, "lockbox file list");
    mvprintw(1, 0, "up/down move   enter selects   q quits");
    if (file_count == 0) {
        mvprintw(3, 2, "(no stored files)");
        refresh();
        return;
    }
    row = 0;
    while (row < file_count) {
        if (row == highlight_index) {
            attron(A_REVERSE);
        }
        mvprintw(row + 3, 2, "%s", names[row]);
        if (row == highlight_index) {
            attroff(A_REVERSE);
        }
        row = row + 1;
    }
    refresh();
}

/*
 * command_screen
 * Show stored names on a terminal screen.
 *
 * return 0 when the user quits
 */
int command_screen(void)
{
    char names[SCREEN_MAX_FILES][NAME_BUFFER_SIZE];
    int file_count;
    int highlight_index;
    int key;

    file_count = collect_stored_names(names, SCREEN_MAX_FILES);
    highlight_index = 0;
    initscr();
    cbreak();
    noecho();
    keypad(stdscr, TRUE);
    while (1) {
        draw_file_list(names, file_count, highlight_index);
        key = getch();
        if (key == 'q' || key == 'Q') {
            break;
        }
        if (key == KEY_UP && highlight_index > 0) {
            highlight_index = highlight_index - 1;
        } else if (key == KEY_DOWN && highlight_index + 1 < file_count) {
            highlight_index = highlight_index + 1;
        }
        else if (key == '\n' || key == KEY_ENTER) {
            if (file_count > 0) {
                mvprintw(file_count + 5, 2, "chosen: %s", names[highlight_index]);
                refresh();
            }
        }
    }
    endwin();
    return 0;
}
```

screen.h

```
/*
 * screen.h
 * CS 375 — lockbox
 *
 * Author: Kevin Tieu
 *
 * Terminal file list. Arrow keys move. q quits.
 */

#ifndef SCREEN_H
#define SCREEN_H

/*
 * command_screen
 * Open a full-terminal file list of the vault.
 *
 * return 0 when the user quits, 1 if setup failed
 */
int command_screen(void);

#endif
```

vault.c

```c
/*
 * vault.c
 * CS 375 — lockbox
 *
 * Author: Kevin Tieu
 *
 * Create the vault folder and build paths inside it.
 */

#include "vault.h"
#include "lockbox.h"
#include "env.h"

#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <sys/stat.h>

/*
 * ensure_vault
 * Create the vault folder if it is not there yet.
 * EEXIST means the folder already exists. That is not an error.
 *
 * return 0 if it worked, 1 if mkdir failed
 */
int ensure_vault(void)
{
    const char *folder;

    folder = vault_directory();
    if (mkdir(folder, 0700) == -1 && errno != EEXIST) {
        perror(folder);
        return 1;
    }
    return 0;
}

/*
 * name_is_safe
 * A stored name must stay inside the vault folder.
 * Reject empty names, slashes, "." and "..".
 *
 * stored_name - name the caller wants to use inside vault/
 * return 1 if the name is usable, 0 if it is not
 */
int name_is_safe(const char *stored_name)
{
    if (stored_name == NULL || stored_name[0] == '\0') {
        return 0;
    }
    if (strchr(stored_name, '/') != NULL) {
        return 0;
    }
    if (strcmp(stored_name, ".") == 0 || strcmp(stored_name, "..") == 0) {
        return 0;
    }
    return 1;
}

/*
 * base_name
 * Return the part after the last slash.
 *
```

```
 * path - path the user typed
 */
const char *base_name(const char *path)
{
    const char *slash;

    if (path == NULL) {
        return "";
    }
    slash = strrchr(path, '/');
    if (slash == NULL) {
        return path;
    }
    return slash + 1;
}

/*
 * join_vault_file
 * Build a path under the vault folder for the given file name.
 *
 * path_buffer - where to write the path
 * path_size   - how many bytes fit in path_buffer
 * file_name   - the file name to append
 * return 0 if it worked, 1 if the path is too long
 */
static int join_vault_file(char *path_buffer, size_t path_size,
                           const char *file_name)
{
    int written;

    written = snprintf(path_buffer, path_size, "%s/%s",
                       vault_directory(), file_name);
    if (written < 0 || (size_t)written >= path_size) {
        fprintf(stderr, "path too long\n");
        return 1;
    }
    return 0;
}

/*
 * make_vault_path
 * Build the full path of one stored file.
 *
 * path_buffer - caller array
 * path_size   - size of that array
 * stored_name - plain file name, no slashes
 * return 0 if it worked, 1 if the name is bad or the path is too long
 */
int make_vault_path(char *path_buffer, size_t path_size,
                    const char *stored_name)
{
    if (!name_is_safe(stored_name)) {
        fprintf(stderr, "bad name: %s\n",
                stored_name != NULL ? stored_name : "(null)");
        return 1;
    }
    return join_vault_file(path_buffer, path_size, stored_name);
}

/*
 * make_log_path
 * Build the path of vault/lockbox.log.
 *
 * return 0 if it worked, 1 if the path is too long
 */
int make_log_path(char *path_buffer, size_t path_size)
{
```

```
    return join_vault_file(path_buffer, path_size, LOG_FILE_NAME);
}

/*
 * is_bookkeeping_name
 * Helper files live in the vault but are not stored files.
 *
 * entry_name - a name from readdir
 * return 1 if this name should be skipped in the file list
 */
int is_bookkeeping_name(const char *entry_name)
{
    if (entry_name == NULL) {
        return 1;
    }
    if (strcmp(entry_name, ".") == 0 || strcmp(entry_name, "..") == 0) {
        return 1;
    }
    if (strcmp(entry_name, LOG_FILE_NAME) == 0) {
        return 1;
    }
    if (strcmp(entry_name, INDEX_FILE_NAME) == 0) {
        return 1;
    }
    if (strcmp(entry_name, INDEX_BIN_NAME) == 0) {
        return 1;
    }
    return 0;
}

/*
 * make_index_path
 * Build the path to the text index file.
 *
 * path_buffer - where to write the path
 * path_size   - how many bytes fit in path_buffer
 * return 0 if it worked, 1 if the path is too long
 */
int make_index_path(char *path_buffer, size_t path_size)
{
    return join_vault_file(path_buffer, path_size, INDEX_FILE_NAME);
}

/*
 * make_index_bin_path
 * Build the path to the binary index file.
 *
 * path_buffer - where to write the path
 * path_size   - how many bytes fit in path_buffer
 * return 0 if it worked, 1 if the path is too long
 */
int make_index_bin_path(char *path_buffer, size_t path_size)
{
    return join_vault_file(path_buffer, path_size, INDEX_BIN_NAME);
}
```

vault.h

```
/*
 * vault.h
 * CS 375 — lockbox
 *
 * Author: Kevin Tieu
 *
 * Vault folder, safe names, and full paths.
 */

#ifndef VAULT_H
#define VAULT_H

#include <stddef.h>

/*
 * ensure_vault
 * Create the vault folder if it is not there yet.
 * EEXIST means the folder already exists. That is not an error.
 *
 * return 0 if it worked, 1 if mkdir failed
 */
int ensure_vault(void);

/*
 * name_is_safe
 * A stored name must stay inside the vault folder.
 * Reject empty names, slashes, "." and "..".
 *
 * stored_name - name the caller wants to use inside vault/
 * return 1 if the name is usable, 0 if it is not
 */
int name_is_safe(const char *stored_name);
/*
 * base_name
 * Return the part after the last slash.
 * put /tmp/notes.txt should store notes.txt, not the whole path.
 *
 * path - path the user typed
 */
const char *base_name(const char *path);
/*
 * make_vault_path
 * Build the full path of one stored file.
 *
 * path_buffer - caller array
 * path_size   - size of that array
 * stored_name - plain file name, no slashes
 * return 0 if it worked, 1 if the name is bad or the path is too long
 */
int make_vault_path(char *path_buffer, size_t path_size,
                    const char *stored_name);
/*
 * make_log_path
 * Build the path of vault/lockbox.log.
 *
 * return 0 if it worked, 1 if the path is too long
 */
int make_log_path(char *path_buffer, size_t path_size);

/*
 * is_bookkeeping_name
 * The log, index, fifo, and stats files live in the vault folder
 * but they are not files the user stored with put.
```

```
 *
 * entry_name - a name from readdir
 * return 1 if this name should be skipped in the file list
 */
int is_bookkeeping_name(const char *entry_name);

int make_index_path(char *path_buffer, size_t path_size);
int make_index_bin_path(char *path_buffer, size_t path_size);

#endif
```

GraphNet

Graph Algorithms and Network Analysis

A Python toolkit implementing core graph algorithms from first principles. Includes an adjacency-list graph, breadth-first and depth-first search, Dijkstra's shortest path, and simple centrality measures.

graph_core.py

```
"""
graph_core.py
Personal project – GraphNet

Author: Kevin Tieu

Pure Python graph data structure for discrete mathematics algorithms.
Supports directed and undirected graphs with weighted edges.
"""

from collections import defaultdict
from typing import Dict, List, Set, Tuple, Optional

class Graph:
    """
    Undirected or directed graph using adjacency lists.
    Supports weighted edges.
    """

    """
    Create an empty graph.

    directed - True for a directed graph, False for undirected
    return nothing
    """
    def __init__(self, directed: bool = False) -> None:
        self.directed = directed
        self.adjacency: Dict[str, Dict[str, float]] = defaultdict(dict)
        self.nodes: Set[str] = set()

    """
    Add an isolated node to the graph.
    The node is recorded even if it has no edges yet.

    node - the name of the node to add
    return nothing
    """
    def add_node(self, node: str) -> None:
        self.nodes.add(node)
        if node not in self.adjacency:
            self.adjacency[node] = {}

    """
    Add a weighted edge from source to target.
    For an undirected graph, the reverse edge is added as well.
    Both nodes are created if they do not already exist.

    source - the node the edge starts from
    target - the node the edge points to
    weight - the cost of the edge (default 1.0)
    return nothing
    """
    def add_edge(self, source: str, target: str, weight: float = 1.0) -> None:
        self.add_node(source)
        self.add_node(target)
        self.adjacency[source][target] = weight
        if not self.directed:
            self.adjacency[target][source] = weight

    """
    Return the nodes that can be reached in one step from the given node.
```

```
node - the node to look up
return a list of neighbor names
"""
def neighbors(self, node: str) -> List[str]:
    return list(self.adjacency.get(node, {}).keys())

"""
Return the weight of the edge from source to target.
If the edge does not exist, return None.

source - the start of the edge
target - the end of the edge
return the weight, or None
"""
def weight(self, source: str, target: str) -> Optional[float]:
    return self.adjacency.get(source, {}).get(target)

"""
Return how many nodes are in the graph.

return the node count
"""
def number_of_nodes(self) -> int:
    return len(self.nodes)

"""
Return how many edges are in the graph.
An undirected edge is counted once.

return the edge count
"""
def number_of_edges(self) -> int:
    count = sum(len(neighbors) for neighbors in self.adjacency.values())
    return count if self.directed else count // 2

"""
Build a printable summary of the graph and its edges.

return a multi-line string listing each node and its edges
"""
def __str__(self) -> str:
    lines = [f"Graph({'directed' if self.directed else 'undirected'})"]
    for node in sorted(self.nodes):
        edges = [f"{neighbor}:{weight}" for neighbor, weight in self.adjacency[node].items()]
        lines.append(f"  {node} -> {', '.join(edges) if edges else '(isolated)'}")
    return "\n".join(lines)
```

algorithms.py

```
"""
algorithms.py
Personal project — GraphNet

Author: Kevin Tieu

Core graph algorithms implemented from first principles.
Includes breadth-first search, depth-first search, Dijkstra,
path reconstruction, degree centrality, and connected components.
"""


from collections import deque, defaultdict
from typing import Dict, List, Optional, Set, Tuple
from graph_core import Graph

"""
Breadth-first search from a start node.
Visits nodes level by level and records the parent of each node so the
shortest unweighted path can be rebuilt later.

graph - the graph to search
start - the node to begin from
return a dictionary mapping each reached node to its parent
"""
def breadth_first_search(graph: Graph, start: str) -> Dict[str, Optional[str]]:
    if start not in graph.nodes:
        raise ValueError(f"Start node '{start}' is not in the graph.")

    parent: Dict[str, Optional[str]] = {start: None}
    queue = deque([start])

    while queue:
        current = queue.popleft()
        for neighbor in graph.neighbors(current):
            if neighbor not in parent:
                parent[neighbor] = current
                queue.append(neighbor)

    return parent

"""
Depth-first search from a start node.
Follows one path as far as it will go before backtracking, and records
every node that can be reached from the start.

graph - the graph to search
start - the node to begin from
return the set of nodes reachable from the start
"""
def depth_first_search(graph: Graph, start: str) -> Set[str]:
    if start not in graph.nodes:
        raise ValueError(f"Start node '{start}' is not in the graph.")

    visited: Set[str] = set()

    def visit(node: str) -> None:
        visited.add(node)
        for neighbor in graph.neighbors(node):
            if neighbor not in visited:
                visit(neighbor)
```

```
    visit(start)
    return visited

"""
Dijkstra's algorithm for single-source shortest paths.
Assumes non-negative edge weights. Builds a distance map and a
parent map so the shortest path to any node can be rebuilt.

graph - the weighted graph to search
start - the node to begin from
return a tuple of (distance dictionary, parent dictionary)
"""
def dijkstra_shortest_path(graph: Graph, start: str) -> Tuple[Dict[str, float], Dict[str,
Optional[str]]]:
    if start not in graph.nodes:
        raise ValueError(f"Start node '{start}' is not in the graph.")

    import heapq
    distances: Dict[str, float] = {node: float('inf') for node in graph.nodes}
    distances[start] = 0.0
    parent: Dict[str, Optional[str]] = {start: None}
    priority_queue = [(0.0, start)]

    while priority_queue:
        current_distance, current = heapq.heappop(priority_queue)
        if current_distance > distances[current]:
            continue

        for neighbor in graph.neighbors(current):
            weight = graph.weight(current, neighbor)
            if weight is None:
                continue
            distance = current_distance + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                parent[neighbor] = current
                heapq.heappush(priority_queue, (distance, neighbor))

    return distances, parent

"""
Rebuild the path from start to end using a parent dictionary
produced by breadth-first search or Dijkstra.

parent - maps each node to the node that led to it
start  - the beginning of the path
end    - the end of the path
return the list of nodes from start to end, or None if no path exists
"""
def reconstruct_path(parent: Dict[str, Optional[str]], start: str, end: str) ->
Optional[List[str]]:
    if end not in parent:
        return None
    path = []
    current = end
    while current is not None:
        path.append(current)
        current = parent.get(current)
    path.reverse()
    return path if path and path[0] == start else None

"""
Degree centrality: number of neighbors divided by (n - 1).
```

```
For a directed graph, uses out-degree.

graph - the graph to measure
return a dictionary mapping each node to its centrality score
"""
def degree_centrality(graph: Graph) -> Dict[str, float]:
    n = graph.number_of_nodes()
    if n <= 1:
        return {node: 0.0 for node in graph.nodes}
    return {node: len(graph.neighbors(node)) / (n - 1) for node in graph.nodes}

"""
Find the connected components of an undirected graph by
repeated depth-first search.

graph - the graph to search
return a list of sets, one set of nodes per component
"""
def connected_components(graph: Graph) -> List[Set[str]]:
    remaining = set(graph.nodes)
    components: List[Set[str]] = []
    while remaining:
        start = remaining.pop()
        component = depth_first_search(graph, start)
        components.append(component)
        remaining -= component
    return components
```

graph_analyzer.py

```
"""
graph_analyzer.py
Personal project — GraphNet

Author: Kevin Tieu

Command-line network analysis tool. Loads an edge list and runs
breadth-first search, depth-first search, Dijkstra, centrality,
and connected-component queries.
return nothing
"""


import argparse
import sys
from typing import Optional
from graph_core import Graph
from algorithms import (
    breadth_first_search,
    depth_first_search,
    dijkstra_shortest_path,
    reconstruct_path,
    degree_centrality,
    connected_components,
)

"""
Load a graph from a simple edge-list file.
Each line is: source target [weight]. Lines starting with # are skipped.
file_path - path to the edge-list file
directed  - True for a directed graph, False for undirected
return the loaded Graph
"""
def load_edge_list(file_path: str, directed: bool = False) -> Graph:

    graph = Graph(directed=directed)
    with open(file_path, "r", encoding="utf-8") as handle:
        for line_number, line in enumerate(handle, start=1):
            line = line.strip()
            if not line or line.startswith("#"):
                continue
            parts = line.split()
            if len(parts) < 2:
                print(f"Warning: skipping malformed line {line_number}: {line}", file=sys.stderr)
                continue
            source, target = parts[0], parts[1]
            weight = float(parts[2]) if len(parts) >= 3 else 1.0
            graph.add_edge(source, target, weight)
    return graph

"""
Read command-line options, load the edge list, and run the
requested graph queries.
return nothing
"""
def main() -> None:

    parser = argparse.ArgumentParser(
        description="Graph algorithms and network analysis toolkit (Kevin Tieu)."
    )
    parser.add_argument("edge_file", help="Path to edge-list file")
    parser.add_argument("--directed", action="store_true", help="Treat the graph as directed")
    parser.add_argument("--start", help="Start node for path algorithms")
```

```
    parser.add_argument("--end", help="End node for path reconstruction")
    parser.add_argument("--algorithm", choices=["bfs", "dfs", "dijkstra", "centrality",
"components"],
                        default="components", help="Algorithm to run")
    args = parser.parse_args()

    try:
        graph = load_edge_list(args.edge_file, directed=args.directed)
    except FileNotFoundError:
        print(f"Error: file not found: {args.edge_file}", file=sys.stderr)
        sys.exit(1)

    print(f"Loaded graph with {graph.number_of_nodes()} nodes and {graph.number_of_edges()}
edges.")
    print(graph)
    print()

    if args.algorithm == "components":
        components = connected_components(graph)
        print(f"Connected components ({len(components)}):")
        for index, component in enumerate(components, start=1):
            print(f"  Component {index}: {sorted(component)}")

    elif args.algorithm == "centrality":
        centrality = degree_centrality(graph)
        print("Degree centrality (sorted):")
        for node, score in sorted(centrality.items(), key=lambda item: item[1], reverse=True):
            print(f"  {node}: {score:.4f}")

    elif args.algorithm in ("bfs", "dfs", "dijkstra"):
        if not args.start:
            print("Error: --start is required for path algorithms.", file=sys.stderr)
            sys.exit(1)
        if args.algorithm == "bfs":
            parent = breadth_first_search(graph, args.start)
            print(f"BFS parent tree from {args.start}:")
            for node, parent_node in sorted(parent.items()):
                print(f"  {node} <- {parent_node}")
            if args.end:
                path = reconstruct_path(parent, args.start, args.end)
                print(f"Path to {args.end}: {' -> '.join(path) if path else 'unreachable'}")

        elif args.algorithm == "dfs":
            reachable = depth_first_search(graph, args.start)
            print(f"DFS reachable nodes from {args.start}: {sorted(reachable)}")

        elif args.algorithm == "dijkstra":
            distances, parent = dijkstra_shortest_path(graph, args.start)
            print(f"Dijkstra distances from {args.start}:")
            for node in sorted(distances):
                dist = distances[node]
                print(f"  {node}: {dist if dist != float('inf') else 'inf'}")
            if args.end:
                path = reconstruct_path(parent, args.start, args.end)
                if path:
                    total = distances[args.end]
                    print(f"Shortest path to {args.end} (distance {total}): {' -> '.join(path)}")
                else:
                    print(f"{args.end} is unreachable from {args.start}.")

if __name__ == "__main__":
    main()
```

Traffic Light

Java Finite State Machine with JUnit

A red-yellow-green traffic light implemented as a finite state machine in Java, with JUnit tests that verify state transitions and timing behavior.

TrafficLight.java

```
package org.example;

/**
 * TrafficLight.java
 * Personal project – Traffic Light
 *
 * Author: Kevin Tieu
 *
 * States and transitions (timer-driven cycle):
 *   RED    + TIMER_EXPIRES -> GREEN
 *   GREEN  + TIMER_EXPIRES -> YELLOW
 *   YELLOW + TIMER_EXPIRES -> RED
 *
 * Any other event leaves the current state unchanged.
 */
public class TrafficLight {

    /** Possible light states. */
    public enum State {
        RED, GREEN, YELLOW
    }

    /** Events that can drive a transition. */
    public enum Event {
        TIMER_EXPIRES
    }

    /**
     * Returns the printable name of a state.
     * @param state the current state
     * @return the state name (e.g., "RED")
     */
    public static String stateName(State state) {
        return state.name();
    }

    /**
     * Computes the next state given the current state and an event.
     * Only TIMER_EXPIRES produces a change; all other events are ignored.
     *
     * @param current the present state
     * @param event   the event that just occurred (may be null)
     * @return the resulting state
     */
    public static State transition(State current, Event event) {
        // Guard: ignore anything that is not a timer expiry
        if (event != Event.TIMER_EXPIRES) {
            return current;
        }

        // Timer-driven cycle
        return switch (current) {
            case RED    -> State.GREEN;
            case GREEN  -> State.YELLOW;
            case YELLOW -> State.RED;
        };
    }
}
```

Main.java

```
package org.example;

import static org.example.TrafficLight.State.*;
import static org.example.TrafficLight.Event.TIMER_EXPIRES;

/**
 * Main.java
 * Simple driver that cycles the traffic light FSM.
 * Author: Kevin Tieu
 */
public class Main {
    /**
     * Start the program. Cycle the light six times with a two-second pause.
     *
     * args - command-line arguments (unused)
     */
    public static void main(String[] args) throws InterruptedException {
        TrafficLight.State current = RED;
        int cycles = 6;

        System.out.println("Traffic light FSM (timer-driven cycle)");
        System.out.println("--------------------------------------");

        for (int i = 0; i < cycles; i++) {
            System.out.println("State: " + TrafficLight.stateName(current));
            Thread.sleep(2000); // 2-second pause between transitions
            current = TrafficLight.transition(current, TIMER_EXPIRES);
        }

        System.out.println("Finished.");
    }
}
```

TrafficLightTest.java

```
package org.example;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.example.TrafficLight.State.*;
import static org.example.TrafficLight.Event.TIMER_EXPIRES;

/**
 * TrafficLightTest.java
 * Personal project – Traffic Light
 *
 * Author: Kevin Tieu
 *
 * Tests were written first (TDD style) and then the transition logic
 * was implemented to satisfy them.
 */
class TrafficLightTest {

    /**
     * RED + timer expires must become GREEN.
     */
    @Test
    void redTimerExpiresGoesToGreen() {
        assertEquals(GREEN, TrafficLight.transition(RED, TIMER_EXPIRES));
    }

    /**
     * GREEN + timer expires must become YELLOW.
     */
    @Test
    void greenTimerExpiresGoesToYellow() {
        assertEquals(YELLOW, TrafficLight.transition(GREEN, TIMER_EXPIRES));
    }

    /**
     * YELLOW + timer expires must become RED.
     */
    @Test
    void yellowTimerExpiresGoesToRed() {
        assertEquals(RED, TrafficLight.transition(YELLOW, TIMER_EXPIRES));
    }

    /**
     * An unrecognized event (represented here by null) must leave the
     * state unchanged.
     */
    @Test
    void unknownEventLeavesStateUnchanged() {
        assertEquals(RED, TrafficLight.transition(RED, null));
    }
}
```

Multi-Connection Chat

GUI Client and Multi-Threaded Server

A chat system with a Windows Forms GUI client and a multi-threaded server. Supports concurrent connections with separate ports for authentication, sending, and receiving messages.

Form1.cs — Login

```
/*
 * Form1.cs
 * CS 346 – Multi-Connection Chat
 *
 * Author: Kevin Tieu
 *
 * Login form. Collects the username and password and connects to the server.
 */

using System;
using System.Collections;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Windows.Forms;

namespace GUIChat
{
    public partial class Form1 : Form
    {
        private const int AuthPort = 3461;
        private const int WritePort = 3462;
        private const int ReadPort = 3463;

        public static string AuthKey = "";
        private static ArrayList messages = new ArrayList();

        public static string InputAddress = "";
        public static string InputName = "";
        public static string InputPass = "";

        /*
         * Form1
         * Build the login form. The address, username, and password boxes
         * are filled in by the designer. This method only starts the form.
         * return nothing
         */
        public Form1()
        {
            InitializeComponent();
        }

        /*
         * CloseButton_Click
         * Close the login form when the user clicks Close.
         *
         * sender - the control that raised the event
         * e      - the event arguments
         * return nothing
         */
        private void CloseButton_Click(object sender, EventArgs e)
        {
            Close();
        }

        /*
         * HelloButton_Click
         * Read the address, username, and password, authenticate with
         * the server, and open the chat window if it succeeds.
         *
         * sender - the control that raised the event
         * e      - the event arguments
```

```
         * return nothing
         */
        public void HelloButton_Click(object sender, EventArgs e)
        {
            // Read the three fields from the form
            InputAddress = AddressText.Text;
            InputName = NameText.Text;
            InputPass = PasswordText.Text;

            try
            {
                // Connect to the authentication port
                using (TcpClient client = new TcpClient(InputAddress, AuthPort))
                using (BufferedStream stream = new BufferedStream(client.GetStream()))
                using (BinaryReader reader = new BinaryReader(stream))
                using (BinaryWriter writer = new BinaryWriter(stream))
                {
                    // Send the username (length first, then the bytes)
                    writer.Write(IPAddress.HostToNetworkOrder(InputName.Length));
                    writer.Write(Encoding.UTF8.GetBytes(InputName));
                    // Send the password the same way
                    writer.Write(IPAddress.HostToNetworkOrder(InputPass.Length));
                    writer.Write(Encoding.UTF8.GetBytes(InputPass));
                    // Read the access key the server sends back
                    int authLength = IPAddress.NetworkToHostOrder(reader.ReadInt32());
                    byte[] authBytes = reader.ReadBytes(authLength);
                    AuthKey = Encoding.UTF8.GetString(authBytes);
                }

                // Authentication succeeded; open chat form
                this.Hide();
                Form2 chatForm = new Form2();
                chatForm.Show();
            }
            catch (Exception ex)
            {
                MessageBox.Show("Connection or authentication failed: " + ex.Message,
                                "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }
    }
}
```

Form2.cs — Chat Window

```
/*
 * Form2.cs
 * CS 346 – Multi-Connection Chat
 *
 * Author: Kevin Tieu
 *
 * Main chat window. Shows messages and sends new ones.
 */

using System;
using System.IO;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading;
using System.Windows.Forms;

namespace GUIChat
{
    public partial class Form2 : Form
    {
        private const int WritePort = 3462;
        private const int ReadPort = 3463;

        private Thread receiveThread;
        private bool keepReceiving;

        /*
         * Form2
         * Build the chat window. Open the receive connection
         * to the server and start showing messages as they arrive.
         * return nothing
         */
        public Form2()
        {
            InitializeComponent();
            UsersOn.Text = Form1.InputName;
            keepReceiving = true;
            receiveThread = new Thread(ReceiveLoop);
            receiveThread.IsBackground = true;
            receiveThread.Start();
        }

        /*
         * textBox1_TextChanged
         * ReplyBox is read-only. Nothing to do when its text changes.
         *
         * sender - the control that raised the event
         * e      - the event arguments
         * return nothing
         */
        private void textBox1_TextChanged(object sender, EventArgs e)
        {
        }

        /*
         * UsersOn_TextChanged
         * UsersOn is read-only. Nothing to do when its text changes.
         *
         * sender - the control that raised the event
         * e      - the event arguments
         * return nothing
         */
```

```
private void UsersOn_TextChanged(object sender, EventArgs e)
{
}

/*
 * CommentBox_TextChanged
 * Enable the post button only when the user has typed something.
 *
 * sender - the control that raised the event
 * e      - the event arguments
 * return nothing
 */
private void CommentBox_TextChanged(object sender, EventArgs e)
{
    ConfirmPost.Enabled = CommentBox.Text.Trim().Length > 0;
}

/*
 * ConfirmPost_Click
 * Send the comment-box text to the server, then clear the box.
 *
 * sender - the control that raised the event
 * e      - the event arguments
 * return nothing
 */
private void ConfirmPost_Click(object sender, EventArgs e)
{
    string message = CommentBox.Text.Trim();
    // Do nothing if the box is empty
    if (message.Length == 0)
    {
        return;
    }

    try
    {
        // Connect to the write port
        using (TcpClient client = new TcpClient(Form1.InputAddress, WritePort))
        using (BufferedStream stream = new BufferedStream(client.GetStream()))
        using (BinaryWriter writer = new BinaryWriter(stream))
        {
            // Send the access key so the server knows who we are
            byte[] keyBytes = Encoding.UTF8.GetBytes(Form1.AuthKey);
            writer.Write(IPAddress.HostToNetworkOrder(keyBytes.Length));
            writer.Write(keyBytes);

            byte[] messageBytes = Encoding.UTF8.GetBytes(message);
            writer.Write(IPAddress.HostToNetworkOrder(messageBytes.Length));
            writer.Write(messageBytes);
            stream.Flush();
        }

        CommentBox.Clear();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Could not send message: " + ex.Message,
                        "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

/*
 * ReceiveLoop
 * Connect to the read port, send the auth key, and append each
 * incoming message to the reply box. Runs on a background thread.
 * return nothing
```

```
         */
        private void ReceiveLoop()
        {
            try
            {
                using (TcpClient client = new TcpClient(Form1.InputAddress, ReadPort))
                using (BufferedStream stream = new BufferedStream(client.GetStream()))
                using (BinaryReader reader = new BinaryReader(stream))
                using (BinaryWriter writer = new BinaryWriter(stream))
                {
                    // Send the access key so the server knows who we are
                    byte[] keyBytes = Encoding.UTF8.GetBytes(Form1.AuthKey);
                    writer.Write(IPAddress.HostToNetworkOrder(keyBytes.Length));
                    writer.Write(keyBytes);
                    stream.Flush();

                    while (keepReceiving)
                    {
                        int nameLength = IPAddress.NetworkToHostOrder(reader.ReadInt32());
                        byte[] nameBytes = reader.ReadBytes(nameLength);
                        string name = Encoding.UTF8.GetString(nameBytes);

                        int messageLength = IPAddress.NetworkToHostOrder(reader.ReadInt32());
                        byte[] messageBytes = reader.ReadBytes(messageLength);
                        string message = Encoding.UTF8.GetString(messageBytes);

                        string line = name + ": " + message + Environment.NewLine;
                        if (ReplyBox.InvokeRequired)
                        {
                            ReplyBox.Invoke(new Action(() => ReplyBox.AppendText(line)));
                        }
                        else
                        {
                            ReplyBox.AppendText(line);
                        }
                    }
                }
            }
            catch (Exception)
            {
                // Connection closed or server stopped. Leave the window open.
            }
        }

        /*
         * OnFormClosing
         * Stop the receive thread when the window closes.
         *
         * e - the close event arguments
         * return nothing
         */
        protected override void OnFormClosing(FormClosingEventArgs e)
        {
            keepReceiving = false;
            base.OnFormClosing(e);
        }
    }
}
```

Server.cs

```
/*
 * Server.cs
 * CS 346 – Multi-Connection Chat
 *
 * Author: Kevin Tieu
 *
 * Core server. Holds the active users and starts the authenticator,
 * receiver, and broadcasters.
 */

using ChatServer.Library.Logging;
using System;
using System.Collections.Generic;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading;

namespace ChatServer.Library
{
    public class Server
    {
        public Dictionary<string, ChatUser> ActiveUsers { get; set; }
        public ILogger Logger
        {
            get
            {
                return _authenticator.Logger;
            }
            set
            {
                _authenticator.Logger = value;
                _receiver.Logger = value;
                foreach (var broadcaster in _broadcasters)
                {
                    broadcaster.Logger = value;
                }
            }
        }

        public int AuthenticationPort
        {
            get
            {
                return _authenticator.ListenPort;
            }
            set
            {
                _authenticator.ListenPort = value;
            }
        }
        public int ReceivePort
        {
            get
            {
                return _receiver.ListenPort;
            }
            set
            {
                _receiver.ListenPort = value;
            }
        }
        public int BroadcastPort { get; set; }
        public int MaxConcurrentUsers { get; private set; }
```

```
private static Server _instance = null;
private Authenticator _authenticator = new Authenticator();
private Receiver _receiver = new Receiver();
private List<Broadcaster> _broadcasters = new List<Broadcaster>();
private Thread _authenticatorThread = null;
private Thread _receiverThread = null;
private List<Thread> _broadcasterThreads = new List<Thread>();

/*
 * Server
 * Build the server object and set how many clients it will accept.
 * The authenticator, receiver, and broadcaster list are created here.
 *
 * maxConcurrentUsers - how many clients can be connected at once
 * return nothing
 */
private Server(int maxConcurrentUsers = 40)
{
    ActiveUsers = new Dictionary<string, ChatUser>();
    MaxConcurrentUsers = maxConcurrentUsers;
    for (int i = 0; i < MaxConcurrentUsers; i++)
    {
        _broadcasters.Add(new Broadcaster());
        _broadcasters[i].Id = i;
        _broadcasters[i].Server = this;
    }

    _authenticator.Server = this;
    _authenticator.ListenPort = AuthenticationPort;

    _receiver.Server = this;
    _receiver.ListenPort = ReceivePort;
}

/*
 * MessageReceived
 * Take a message from one client and hand it to every broadcaster
 * so the other clients can see it.
 *
 * user    - the client that sent the message
 * message - the text that was received
 * return nothing
 */
public void MessageReceived(ChatUser user, string message)
{
    foreach (var broadcaster in _broadcasters)
    {
        broadcaster.AddMessage(user, message);
    }
}

/*
 * Run
 * Start the authenticator and the receiver on their own threads.
 * One broadcaster thread is started for each allowed user.
 * This call does not return until the server is stopped.
 */
public void Run()
{
    ThreadStart authStart = _authenticator.Run;
    _authenticatorThread = new Thread(authStart);
    _authenticatorThread.Start();

    ThreadStart receiverStart = _receiver.Run;
    _receiverThread = new Thread(receiverStart);
    _receiverThread.Start();
```

```
            TcpListener listener = new TcpListener(IPAddress.Any, BroadcastPort);
            listener.Start();
            foreach (var broadcaster in _broadcasters)
            {
                broadcaster.Listener = listener;
                ThreadStart broadcastStart = broadcaster.Run;
                Thread current = new Thread(broadcastStart);
                current.Start();
                _broadcasterThreads.Add(current);
            }
        }

        /*
         * GetInstance
         * Return the one shared Server object. Create it the first time
         * this is called.
         *
         * return the Server instance
         */
        public static Server GetInstance()
        {
            if (_instance == null)
            {
                _instance = new Server();
            }
            return _instance;
        }
    }
}
```

Authenticator.cs

```
/*
 * Authenticator.cs
 * CS 346 – Multi-Connection Chat
 *
 * Author: Kevin Tieu
 *
 * Listens for new clients, checks the username and password, and registers them.
 */

using ChatServer.Library.Logging;
using System;
using System.Collections.Generic;
using System.IO;
using System.Net;
using System.Net.Sockets;
using System.Security.Cryptography;
using System.Text;

namespace ChatServer.Library
{
    public class Authenticator
    {
        public int ListenPort { get; set; }
        public bool IsRunning { get; private set; }
        public ILogger Logger { get; set; }
        public Server Server { get; set; }

        /*
         * Authenticator
         * Create the authenticator. The listen port and server
         * are set by the caller before Run is called.
         * return nothing
         */
        public Authenticator()
        {
        }

        /*
         * Run
         * Listen on the authentication port. For each new client, read the
         * username and password, check them, and add the user to the active list.
         * Keep doing this until the server is stopped.
         * return nothing
         */
        public void Run()
        {
            IsRunning = true;
            TcpListener listener = new TcpListener(IPAddress.Any, ListenPort);
            listener.Start();
            while (IsRunning)
            {
                Logger.Log("Authenticator waiting for connection on port {0}...", ListenPort);
                TcpClient client = null;
                try
                {
                    client = listener.AcceptTcpClient();

#if DEBUG == false
                    //timeouts affect debugging when stepping through code
                    client.ReceiveTimeout = 5000;
#endif

                }
```

```
                catch (Exception ex)
                {
                    Logger.Log("Could not accept TCP client: {0}", ex.Message);
                    continue;
                }
                Logger.Log("Authenticator accepting client: {0}", client.Client.RemoteEndPoint);

                BinaryReader reader = null;
                BinaryWriter writer = null;
                try
                {
                    BufferedStream stream = new BufferedStream(client.GetStream());
                    reader = new BinaryReader(stream);
                    writer = new BinaryWriter(stream);
                    int userNameLength = IPAddress.NetworkToHostOrder(reader.ReadInt32());
                    byte[] userNameBytes = reader.ReadBytes(userNameLength);
                    string userName = Encoding.UTF8.GetString(userNameBytes);

                    int passwordLength = IPAddress.NetworkToHostOrder(reader.ReadInt32());
                    byte[] passwordBytes = reader.ReadBytes(passwordLength);
                    string password = Encoding.UTF8.GetString(passwordBytes);

                    //TODO validate
                    string address =
((IPEndPoint)client.Client.RemoteEndPoint).Address.ToString();
                    ChatUser user = new ChatUser() {
                        Address = address,
                        AuthString = GenerateAccessKey(),
                        Name = userName
                    };
                    Server.ActiveUsers[user.AuthString] = user;
                    writer.Write(IPAddress.HostToNetworkOrder(user.AuthString.Length));
                    writer.Write(Encoding.UTF8.GetBytes(user.AuthString));

                }
                catch (Exception ex)
                {
                    Logger.Log("Authenticator exception handling client {0}, {1}: ",
client.Client.RemoteEndPoint, ex.Message);
                }
                finally
                {
                    Logger.Log("Authenticator done handling client {0}",
client.Client.RemoteEndPoint);
                    reader.Close();
                    writer.Close();
                    if (client != null)
                    {
                        client.Close();
                    }
                }
            }
        }

        /*
         * GenerateAccessKey
         * Build a random access key for a newly authenticated user.
         *
         * return the key as a string
         */
        public static string GenerateAccessKey()
        {
            var key = new byte[32];
            using (var generator = RandomNumberGenerator.Create())
            {
                generator.GetBytes(key);
            }
```

```
            return Convert.ToBase64String(key);
        }
    }
}
```

Receiver.cs

```
/*
 * Receiver.cs
 * CS 346 – Multi-Connection Chat
 *
 * Author: Kevin Tieu
 *
 * Reads messages from connected clients and passes them to the server.
 */

using ChatServer.Library.Logging;
using System;
using System.Collections.Generic;
using System.IO;
using System.Net;
using System.Net.Sockets;
using System.Text;

namespace ChatServer.Library
{
    public class Receiver
    {
        public int ListenPort { get; set; }
        public bool IsRunning { get; private set; }
        public ILogger Logger { get; set; }
        public Server Server { get; set; }

        /*
         * Receiver
         * Create the receiver. The listen port and server
         * are set by the caller before Run is called.
         * return nothing
         */
        public Receiver()
        {

        }

        /*
         * Run
         * Listen on the receive port. When a client sends a line of text,
         * pass that line to the server so it can be broadcast to the others.
         * Keep doing this until the server is stopped.
         * return nothing
         */
        public void Run()
        {

            IsRunning = true;
            TcpListener listener = new TcpListener(IPAddress.Any, ListenPort);
            listener.Start();
            while (IsRunning)
            {
                Logger.Log("Receiver waiting for connection on port {0}...", ListenPort);
                TcpClient client = null;
                try
                {
                    client = listener.AcceptTcpClient();

#if DEBUG == false
                    //timeouts affect debugging when stepping through code
                    client.ReceiveTimeout = 5000;
#endif
```

```
                }
                catch (Exception ex)
                {
                    Logger.Log("Could not accept TCP client: {0}", ex.Message);
                    continue;
                }
                Logger.Log("Receiver accepting client: {0}", client.Client.RemoteEndPoint);

                BinaryReader reader = null;
                BinaryWriter writer = null;
                try
                {
                    BufferedStream stream = new BufferedStream(client.GetStream());
                    reader = new BinaryReader(stream);
                    writer = new BinaryWriter(stream);
                    int authKeyLength = IPAddress.NetworkToHostOrder(reader.ReadInt32());
                    byte[] authKeyBytes = reader.ReadBytes(authKeyLength);
                    string authKey = Encoding.UTF8.GetString(authKeyBytes);

                    if(Server.ActiveUsers.ContainsKey(authKey) == true)
                    {
                        int messageLength = IPAddress.NetworkToHostOrder(reader.ReadInt32());
                        byte[] messageBytes = reader.ReadBytes(messageLength);
                        string message = Encoding.UTF8.GetString(messageBytes);
                        Server.MessageReceived(Server.ActiveUsers[authKey], message);

                        Logger.Log("{0} says: {1}", Server.ActiveUsers[authKey].Name, message);
                    }

                }
                catch (Exception ex)
                {
                    Logger.Log("Receiver exception handling client {0}, {1}: ",
client.Client.RemoteEndPoint, ex.Message);
                }
                finally
                {
                    Logger.Log("Receiver done handling client {0}", client.Client.RemoteEndPoint);
                    reader.Close();
                    writer.Close();
                }
            }
        }
    }
}
```

Broadcaster.cs

```
/*
 * Broadcaster.cs
 * CS 346 – Multi-Connection Chat
 *
 * Author: Kevin Tieu
 *
 * Sends messages out to the connected clients.
 */

using ChatServer.Library.Logging;
using System;
using System.Collections.Generic;
using System.IO;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading;

namespace ChatServer.Library
{
    public class Broadcaster
    {
        public int Id { get; set; }
        public ILogger Logger { get; set; }
        private Queue<KeyValuePair<ChatUser, string>> PendingMessages { get; set; }
        public Server Server { get; set; }
        public bool IsRunning { get; set; }
        public TcpListener Listener { get; set; }

        /*
         * Broadcaster
         * Create a broadcaster. The listener and server are set
         * by the caller before Run is called.
         * return nothing
         */
        public Broadcaster()
        {
            PendingMessages = new Queue<KeyValuePair<ChatUser, string>>();
        }

        /*
         * AddMessage
         * Queue a message so it will be sent to the connected client.
         *
         * user    - the client that sent the message
         * message - the text to send
         * return nothing
         */
        public void AddMessage(ChatUser user, string message)
        {
            PendingMessages.Enqueue(new KeyValuePair<ChatUser, string>(user, message));
        }

        /*
         * Run
         * Wait for messages that need to go out, then write each one to
         * the clients that should receive it. Keep doing this until the
         * server is stopped.
         * return nothing
         */
        public void Run()
        {
            IsRunning = true;
```

```
            while (IsRunning)
            {
                Logger.Log("Broadcaster {0} waiting for connection...", Id);
                TcpClient client = null;
                try
                {
                    client = Listener.AcceptTcpClient();

#if DEBUG == false
                    //timeouts affect debugging when stepping through code
                    client.ReceiveTimeout = 5000;
#endif

                }
                catch (Exception ex)
                {
                    Logger.Log("Broadcaster {0} could not accept TCP client: {1}", Id,
ex.Message);
                    continue;
                }
                Logger.Log("Broadcaster {0} accepting client: {1}", Id,
client.Client.RemoteEndPoint);

                BinaryReader reader = null;
                BinaryWriter writer = null;
                try
                {
                    BufferedStream stream = new BufferedStream(client.GetStream());
                    reader = new BinaryReader(stream);
                    writer = new BinaryWriter(stream);
                    int authKeyLength = IPAddress.NetworkToHostOrder(reader.ReadInt32());
                    byte[] authKeyBytes = reader.ReadBytes(authKeyLength);
                    string authKey = Encoding.UTF8.GetString(authKeyBytes);

                    if (Server.ActiveUsers.ContainsKey(authKey) == true)
                    {
                        while(true)
                        {
                            if(PendingMessages.Count > 0)
                            {
                                var message = PendingMessages.Dequeue();
                                byte[] userBytes = Encoding.UTF8.GetBytes(message.Key.Name);
                                writer.Write(IPAddress.HostToNetworkOrder(userBytes.Length));
                                writer.Write(userBytes);

                                byte[] messageBytes = Encoding.UTF8.GetBytes(message.Value);
                                writer.Write(IPAddress.HostToNetworkOrder(messageBytes.Length));
                                writer.Write(messageBytes);
                                stream.Flush();
                            }
                            else
                            {
                                Thread.Sleep(500);
                            }
                        }
                    }

                }
                catch (Exception ex)
                {
                    Logger.Log("Broadcaster {0} exception handling client {1}, {2}: ", Id,
client.Client.RemoteEndPoint, ex.Message);
                }
                finally
                {
```

```
                    Logger.Log("Broadcaster {0} done handling client {1}", Id,
client.Client.RemoteEndPoint);
                    reader.Close();
                    writer.Close();
                    if (client != null)
                    {
                        client.Close();
                    }
                }
            }
        }
    }
}
```

ChatUser.cs

```
/*
 * ChatUser.cs
 * CS 346 – Multi-Connection Chat
 *
 * Author: Kevin Tieu
 *
 * Represents one connected user and their network streams.
 */

using System;
using System.Collections.Generic;
using System.Text;

namespace ChatServer.Library
{
    public class ChatUser
    {
        public string Name { get; set; }
        public string Address { get; set; }
        public string AuthString { get; set; }
    }
}
```